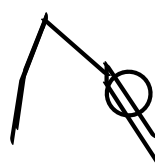
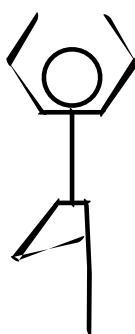


CLOSET

Three-dimensional articulated figures for Typst



Version 0.1.0

Contents

1. Overview	3
1.1. Installation	3
1.2. First figure	3
2. Three-dimensional model	3
2.1. Constructing a pose manually	4
2.2. Composing gesture modifiers	4
2.3. Built-in poses	5
2.4. Controlled variation	5
3. Hand gestures	7
3.1. Canonical gestures	7
3.2. Volumetric tube rendering	7
3.3. Defining a hand pose	8
3.4. Angles, landmarks, and variation	8
4. Camera and projection	9
4.1. One pose, three views	9
4.2. Choosing a useful viewpoint	10
4.3. Framing and focal length	10
5. Calligraphic rendering	10
6. Imported OpenSim motion	11
7. API reference	11
7.1. hand, hand-pose, and hand-gestures	11
7.2. hand-angles, hand-joints, and vary-hand	11
7.3. pose	12
7.4. Gesture modifiers	12
7.5. human	12
7.6. vary-pose	12
7.7. camera	12
7.8. project	12
7.9. joints	12
7.10. pose-from-degrees and constrain-pose	12
8. Troubleshooting	12

1. Overview

Closet draws compact human figures from an articulated model. A pose contains three-dimensional bone directions. The package evaluates those directions into joint positions, projects the joints through a perspective camera, and sends the resulting paths to Premetadated's calligraphic pen engine.

The pipeline is:

```
3D pose directions
-> 3D forward kinematics
-> perspective camera
-> 2D projected paths
-> calligraphic nib envelopes
-> CeTZ vector output
```

This separation is important. Changing the camera does not alter the body or its pose; it only changes how the same 3D joints appear on the page.

1.1. Installation

For local package use, install both repositories under Typst's local package directory:

```
{data-dir}/typst/packages/local/closet/0.1.0
{data-dir}/typst/packages/local/premetadated/0.1.0
```

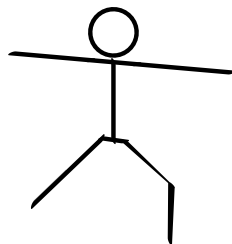
On macOS, {data-dir} is ~/Library/Application Support. Import the package as:

```
#import "@local/closet:0.1.0": human
```

From this repository, examples can instead use `#import "lib.typ"`.

1.2. First figure

```
#import "@local/closet:0.1.0": human
#human(pose: "yoga-warrior-two", scale: 1.2)
```



*Warrior two with the default
camera and pen*

`human` returns a CeTZ canvas, so it can be placed in normal Typst grids, figures, boxes, and alignment containers.

2. Three-dimensional model

Closet uses right-handed (x, y, z) coordinates:

- x runs horizontally across a front-facing body;
- y carries depth, toward or away from the viewer;
- z points upward.

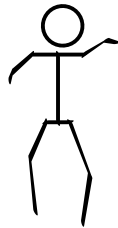
A canonical pose stores `torso` and `side` directions plus one direction for each upper and lower arm and leg. An optional `head` direction lets the head move independently; when omitted, it follows the torso.

Directions need not be unit length because the kinematics normalizes them before applying fixed anatomical segment lengths.

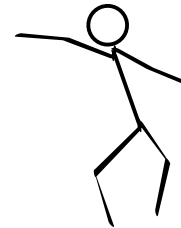
2.1. Constructing a pose manually

Use `pose(...)` to construct a complete pose. Every argument has a standing default, so specify only the directions that differ. This example makes a twisting crouch with one arm reaching forward and the other moving back:

```
#let custom = closet.pose(  
  torso: (0.0, -0.32, 0.95),  
  head: (0.15, -0.2, 0.97),  
  left-upper-arm: (0.35, -0.85, 0.4),  
  left-lower-arm: (0.1, -1.0, 0.1),  
  right-upper-arm: (-0.45, 0.75, -0.5),  
  right-lower-arm: (-0.15, 0.9, -0.4),  
  left-upper-leg: (0.3, -0.7, -0.65),  
  left-lower-leg: (-0.1, 0.25, -0.97),  
  right-upper-leg: (-0.3, 0.5, -0.82),  
  right-lower-leg: (0.1, -0.2, -0.98),  
)  
  
#closet.human(pose: custom)
```



Front: eye (0, -8, 2.4)



Side: eye (8, 0, 2.4)

Each arm and leg direction is local to its segment: the lower-arm vector starts at the elbow, and the lower-leg vector starts at the knee. Positive z points upward. Test a new pose from several cameras because depth components can be hidden in a single projection.

`joints(pose)` evaluates a pose without drawing it. It returns `pelvis`, `neck`, and `head` points plus three-point chains for each arm and leg. Every returned point is a 3D tuple.

2.2. Composing gesture modifiers

Gesture helpers accept either a built-in name or a pose dictionary, and return a new pose dictionary. They can therefore be nested without changing the original. Here a standing figure takes a step, leans, extends the right arm, waves, and tilts its head:

```
#let greeting = closet.tilt-head(  
  closet.wave-arms(  
    closet.stretch-arms(  
      closet.bend-torso(  
        closet.walk-legs("standing", amount: 0.45),  
        amount: 0.12,  
      ),  
      amount: 0.5,  
      side: "right",  
    ),  
    amount: 0.85,  
    side: "right",  
  ),  
)
```

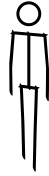
```

    amount: -0.18,
  )
  #closet.human(pose: greeting)

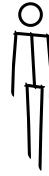
```



Standing



Walk legs



Bend + stretch



Wave + tilt

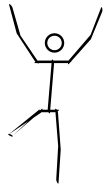
raise-arms, stretch-arms, spread-legs, and wave-arms use an amount from 0 to 1. tilt-head, bend-torso, and walk-legs accept signed values from -1 to 1; changing the sign reverses the tilt, bend, or stride. Arm helpers accept side: "left", "right", or "both". These helpers blend 3D directions and preserve segment lengths, but they are posing conveniences, not collision detection or physiological validation.

2.3. Built-in poses

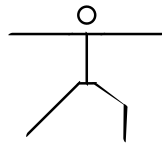
The package includes standing, sitting, cross-legged sitting, thinking, walking, running, downward dog, child's pose, warrior one, warrior two, tree pose, squatting, push-up, pull-up, walking on hands, jumping on one foot, and jumping with open legs. Their stable names are available as keys in `poses`; see `assets/previews/poses.png` for the visual index.



Thinking



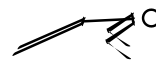
One-foot jump



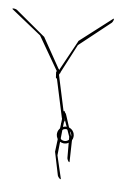
Warrior two



Tree



Push-up



Hand walk

2.4. Controlled variation

`vary-pose(pose, variation:, seed:)` adds small deterministic offsets to the canonical 3D bone directions. It accepts either a built-in pose name or a pose dictionary and returns a new dictionary suitable for `human`, `joints`, or additional editing.

```

#let people = range(5).map(seed =>
  vary-pose("walking", variation: 0.09, seed: seed)
)

#for person in people {
  human(pose: person, camera: profile-camera)
}

```

The same operation is available directly on `human`:

```

#human(pose: "walking", variation: 0.09, seed: 3)

```



Seed 0



Seed 1



Seed 2



Seed 3



Seed 4

The integer seed makes output reproducible. Equal inputs produce identical figures, while different seeds redistribute each limb slightly. Variation is bounded to `0..0.35`; `0` returns the canonical pose, and `0.04..0.12` usually gives useful scene diversity without changing the gesture. Torso and body-side axes receive a smaller perturbation than limbs. All directions are normalized after perturbation, so bone lengths remain fixed.

3. Hand gestures

The hand model follows the standard 21-landmark topology: one wrist, four thumb landmarks, and four landmarks for each finger. It is a small procedural model, not a general anatomical simulator. Canonical gestures store finger curl and spread, thumb opposition, global spread, and wrist orientation. Closet expands those controls into constrained joint angles and fixed-length 3D bones, then uses the same camera projection and Premetadated pen as human.

3.1. Canonical gestures



Open palm



Fist



Pointing



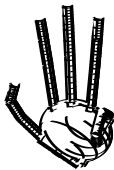
V / peace



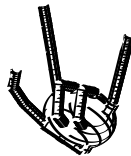
Thumbs-up



Pinch / OK



Three fingers



Rock



Beckoning

```
#hand(  
  gesture: "peace",  
  handedness: "right",  
  camera: camera(  
    eye: (2.4, -7.0, 3.0),  
    target: (0.0, 0.0, 0.65),  
    focal-length: 6.5,  
  ),  
)
```

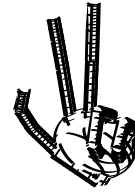
The built-in names are open-palm, fist, pointing, peace, thumbs-up, pinch, three-fingers, rock, and beckoning; ok aliases pinch. Set handedness: "left" to mirror the complete 3D skeleton before wrist rotation.

3.2. Volumetric tube rendering

The default render: "tube" sends each 3D finger chain and thicker metacarpal paths through Premetadated's tube renderer, then adds an oblate ellipsoid for the palm. The solids currently overlap with visible seams; a future Larnt-level union can remove those boundaries. Use render: "skeleton" to inspect the projected landmark centerlines. Sparse light-directed hatching distinguishes phalanges and depth without stippling; set shading: false for outlines only. Both modes use the same gesture, landmarks, handedness, and camera.



Projected skeleton



Volumetric tubes

```
#hand(
  gesture: "peace",
  render: "tube",
  tube-radius: 0.05,
  tube-sides: 10,
  shading: true,
  camera: view,
)
```

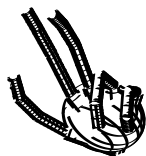
`tube-radius` is measured in the hand model's 3D units. `tube-sides` controls cross-section smoothness and must be at least six. `shading` enables sparse line hatching on the tubes and palm; it never uses stippling. The tube renderer derives its field of view from Closet's camera focal length, so changing `eye`, `target`, `up`, or `focal-length` works consistently in both modes.

3.3. Defining a hand pose

`hand-pose(...)` supplies straight-finger defaults. Finger curl runs from 0 to 1; per-finger spread runs from -1 to 1; global spread runs from 0 to 1.5. Thumb opposition runs from 0 to 1. Wrist pitch and yaw are limited to $\pm 45^\circ$, and roll to $\pm 90^\circ$.

```
#let custom = hand-pose(
  thumb: (curl: 0.35, spread: 0.1, opposition: 0.8),
  index: (curl: 0.05, spread: -0.35),
  middle: (curl: 0.2, spread: 0.15),
  ring: (curl: 0.75, spread: 0.1),
  pinky: (curl: 0.9, spread: 0.2),
  spread: 0.7,
  wrist: (pitch: 8deg, yaw: -12deg, roll: 5deg),
)

#hand(gesture: custom)
```



Right hand



Mirrored left hand

3.4. Angles, landmarks, and variation

`hand-angles(gesture, variation:, seed:)` exposes the expanded MCP/PIP/DIP angles. The four long fingers use maximum flexions of 70° , 95° , and 65° ; thumb CMC/MCP/IP maxima are 40° , 55° , and 60° . Because all three joints derive from one varied curl value, their motion remains correlated.

`hand-joints(gesture, variation:, seed:, handedness:)` returns the 21 named 3D points: `wrist`; `thumb-cmc`, `thumb-mcp`, `thumb-ip`, `thumb-tip`; and `MCP`, `PIP`, `DIP`, and `tip` points for index, middle, ring, and pinky. `hand-topology` exposes the five connected chains. This matches MediaPipe's topology and terminology, without adding MediaPipe or an ML runtime dependency.

```
#let landmarks = hand-joints("pointing")
#let variants = range(5).map(seed =>
  vary-hand("pointing", variation: 0.07, seed: seed)
)
```



Seed 0



Seed 1



Seed 2



Seed 3



Seed 4

Variation is deterministic and bounded to $0..0.25$. It perturbs each finger's shared curl, lateral spread, thumb opposition, and wrist orientation, then clamps every value before angle expansion. Segment lengths never change.

4. Camera and projection

Create a perspective camera with:

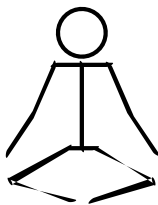
```
#let view = camera(
  eye: (3.8, -8.0, 2.8),
  target: (0.0, 0.0, 0.2),
  up: (0.0, 0.0, 1.0),
  focal-length: 7.0,
)
```

`eye` is the camera position. `target` is the world-space point at the center of the image. `up` establishes image vertical. `focal-length` controls perspective magnification: increasing it enlarges the projected figure when camera distance is unchanged.

The implementation constructs a look-at basis from these values. For point p , eye e , forward vector f , camera-right vector r , and camera-up vector u , depth is $d = (p - e) \cdot f$. Projection is proportional to $((p - e) \cdot \frac{r}{d}, (p - e) \cdot \frac{u}{d})$. Points must remain in front of the camera.

4.1. One pose, three views

The following three figures all use exactly `poses.sitting-cross-legged`. Only the camera position changes.



Front



Three-quarter



Side

The front view emphasizes bilateral symmetry. The three-quarter view reveals which knees and hands advance in depth. The side view compresses the body's width but exposes front-to-back placement. This is why a pose that looks ambiguous from one direction can become immediately legible from another.

```
#let pose = poses.sitting-cross-legged

#human(pose: pose, camera: camera(eye: (0, -8, 2.4)))
#human(pose: pose, camera: camera(eye: (5, -7, 3.0)))
#human(pose: pose, camera: camera(eye: (8, 0, 2.4)))
```

4.2. Choosing a useful viewpoint

Use a front view for poses whose defining feature is left-right symmetry, such as warrior two, tree pose, squatting, and jumping. Use a profile view for poses whose silhouette depends on forward motion or floor contact, such as sitting, walking, running, downward dog, child's pose, and push-up. A three-quarter view is useful when depth relationships matter but both sides must remain visible.

The supplied `front-camera`, `profile-camera`, and `three-quarter-camera` are convenient defaults, not semantic constraints. A pose may face another world direction, and a custom `camera` can always be more informative.

4.3. Framing and focal length

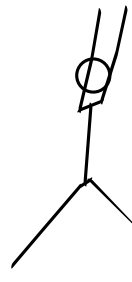
Move `eye` and `target` to frame the subject. Keep `target` near the torso for upright poses and lower it for floor poses. If the figure is too small, increase `focal-length`, increase `human(scale:)`, or move the eye closer. Changing focal length alters perspective and magnification; changing `scale` only changes the final drawing size and pen dimensions.



Focal length 4.5



Focal length 7



Focal length 10

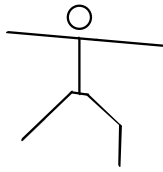
5. Calligraphic rendering

Every bone and the head outline are rendered by `premetadated.stroke.nib-stroke`. The default tangent-following nib has a broad axis of `0.050`, a fine axis of `0.015`, and an offset of `24deg`. Nib dimensions scale with the figure.

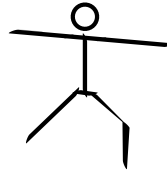
Pass any `Premetadated` pen dictionary to override it:

```
#human(
  pose: "yoga-warrior-two",
  pen: (
    mode: "calligraphic",
    offset: 12deg,
    samples: ((arclength: 0, a: 0.055, b: 0.009),),
  ),
  stroke: rgb("352b25"),
)
```

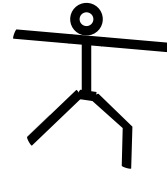
Larger separation between `a` and `b` creates stronger contrast between broad and fine directions. The nib offset rotates that contrast relative to each path's tangent.



Fine



Broad



Rotated

6. Imported OpenSim motion

`tools/import_opensim.py` is an offline adapter. It downloads pinned Gait2354 model and inverse-kinematics data, then writes compact model ranges and paired gait frames to `data/gait2354.json`.

```
python3 tools/import_opensim.py
typst compile --root . examples/opensim.typ build/opensim.pdf
```

`pose-from-degrees(base, values)` converts imported sagittal hip and knee angles into 3D leg directions. Use a canonical 3D pose as the upper-body base:

```
#let data = json("data/gait2354.json")
#let sample = data.samples.first()
#let pose = pose-from-degrees(poses.walking, sample.pose_degrees)
#human(pose: pose, limits: data.limits, camera: profile-camera)
```



right leg forward at 0.6833333 s



left leg forward at 1.3 s

The synchronized hip and knee values belong to one observed time step. Joint ranges alone do not guarantee a realistic combined pose, and the Gait2354 ranges are model domains rather than universal physiological safety limits.

7. API reference

7.1. hand, hand-pose, and hand-gestures

`hand(gesture:, variation:, seed:, handedness:, render:, tube-radius:, tube-sides:, shading:, scale:, stroke:, pen:, camera:)` renders a procedural hand. `render` accepts "skeleton" or "tube". `hand-pose(...)` creates compact finger controls, and `hand-gestures` contains the canonical dictionaries.

7.2. hand-angles, hand-joints, and vary-hand

`hand-angles(gesture, variation:, seed:)` expands controls into constrained angles. `hand-joints(gesture, variation:, seed:, handedness:)` evaluates the 21 3D landmarks. `vary-hand(gesture, variation:, seed:)` returns a reproducibly perturbed control dictionary. `hand-topology` and `hand-joint-limits` expose the connectivity and limits used by the model.

7.3. pose

`pose(torso:, side:, head:, left-upper-arm:, left-lower-arm:, right-upper-arm:, right-lower-arm:, left-upper-leg:, left-lower-leg:, right-upper-leg:, right-lower-leg:)` constructs a complete pose dictionary. All arguments are optional and default to standing. `head: none` follows the torso direction.

7.4. Gesture modifiers

`tilt-head`, `bend-torso`, `raise-arms`, `stretch-arms`, `spread-legs`, `walk-legs`, and `wave-arms` each return a modified copy of a named or custom pose. Functions that affect arms accept `side:` where appropriate. Their results can be passed to another modifier, `human`, `joints`, or `vary-pose`.

7.5. human

`human(pose:, variation:, seed:, scale:, stroke:, pen:, limits:, camera:)` renders one figure. `pose` accepts a built-in name or complete 3D pose dictionary. `stroke` is the ink color. `pen` accepts a `Premetadated` pen; `none` selects the package default. `limits` optionally clamps imported OpenSim leg angles. `camera` controls the projection.

7.6. vary-pose

`vary-pose(pose, variation:, seed:)` returns a reproducibly perturbed 3D pose. The accepted variation range is 0..0.35 and the seed must be an integer.

7.7. camera

`camera(eye:, target:, up:, focal-length:)` validates and returns a camera dictionary. The package exports `default-camera`, `front-camera`, `profile-camera`, and `three-quarter-camera`.

7.8. project

`project(point, camera:)` maps one 3D point to a 2D pair. It is useful when labels or other graphics must align with a figure joint.

7.9. joints

`joints(pose)` evaluates forward kinematics and returns the 3D body landmarks without rendering.

7.10. pose-from-degrees and constrain-pose

These helpers bridge compact OpenSim-derived data to the 3D renderer. They are primarily intended for generated sagittal gait samples.

8. Troubleshooting

- **Package not found:** install both `closet:0.1.0` and `premetadated:0.1.0` in the same local package namespace.
- **A pose looks flat:** move the camera away from a principal axis, for example from (0, -8, 2.4) to (4, -7, 3).
- **A limb is hidden:** rotate the camera around `target`; do not modify the pose merely to compensate for one viewpoint.
- **The figure is clipped:** move `eye` farther away or reduce `scale`.
- **Projection reports a point behind the camera:** move `eye` away from the model and keep `target` between the camera and figure.
- **Calligraphy is too subtle:** increase nib axis `a` or reduce `b`.